

Chapter 13. Fast Fourier Transform

The discrete Fourier transform (DFT) plays an important role in many scientific and technical applications, including time series and waveform analysis, solutions to linear partial differential equations, convolution, digital signal processing, and image filtering. The DFT is a linear transformation that maps n regularly sampled points from a cycle of a periodic signal, like a sine wave, onto an equal number of points representing the frequency spectrum of the signal. In 1965, Cooley and Tukey devised an algorithm to compute the DFT of an n -point series in $O(n \log n)$ operations. Their new algorithm was a significant improvement over previously known methods for computing the DFT, which required $O(n^2)$ operations. The revolutionary algorithm by Cooley and Tukey and its variations are referred to as the **fast Fourier transform** (FFT). Due to its wide application in scientific and engineering fields, there has been a lot of interest in implementing FFT on parallel computers.

Several different forms of the FFT algorithm exist. This chapter discusses its simplest form, the one-dimensional, unordered, radix-2 FFT. Parallel formulations of higher-radix and multidimensional FFTs are similar to the simple algorithm discussed in this chapter because the underlying ideas behind all sequential FFT algorithms are the same. An ordered FFT is obtained by performing bit reversal ([Section 13.4](#)) on the output sequence of an unordered FFT. Bit reversal does not affect the overall complexity of a parallel implementation of FFT.

In this chapter we discuss two parallel formulations of the basic algorithm: the **binary-exchange algorithm** and the **transpose algorithm**. Depending on the size of the input n , the number of processes p , and the memory or network bandwidth, one of these may run faster than the other.

13.1 The Serial Algorithm

Consider a sequence $X = \langle X[0], X[1], \dots, X[n-1] \rangle$ of length n . The discrete Fourier transform of the sequence X is the sequence $Y = \langle Y[0], Y[1], \dots, Y[n-1] \rangle$, where **Equation 13.1**

$$Y[i] = \sum_{k=0}^{n-1} X[k] \omega^{ki}, \quad 0 \leq i < n.$$

In [Equation 13.1](#), ω is the primitive n th root of unity in the complex plane; that is, $\omega = e^{2\pi\sqrt{-1}/n}$, where e is the base of natural logarithms. More generally, the powers of ω in the equation can be thought of as elements of the finite commutative ring of integers modulo n . The powers of ω used in an FFT computation are also known as **twiddle factors**.

The computation of each $Y[i]$ according to [Equation 13.1](#) requires n complex multiplications. Therefore, the sequential complexity of computing the entire sequence Y of length n is $O(n^2)$. The fast Fourier transform algorithm described below reduces this complexity to $O(n \log n)$.

Assume that n is a power of two. The FFT algorithm is based on the following step that permits an n -point DFT computation to be split into two $(n/2)$ -point DFT computations: **Equation 13.2**

$$\begin{aligned} Y[i] &= \sum_{k=0}^{(n/2)-1} X[2k] \omega^{2ki} + \sum_{k=0}^{(n/2)-1} X[2k+1] \omega^{(2k+1)i} \\ &= \sum_{k=0}^{(n/2)-1} X[2k] e^{2(2\pi\sqrt{-1}/n)ki} + \sum_{k=0}^{(n/2)-1} X[2k+1] \omega^i e^{2(2\pi\sqrt{-1}/n)ki} \\ &= \sum_{k=0}^{(n/2)-1} X[2k] e^{2\pi\sqrt{-1}ki/(n/2)} + \omega^i \sum_{k=0}^{(n/2)-1} X[2k+1] e^{2\pi\sqrt{-1}ki/(n/2)} \end{aligned}$$

Let $\tilde{\omega} = e^{2\pi\sqrt{-1}/(n/2)} = \omega^2$; that is, $\tilde{\omega}$ is the primitive $(n/2)$ th root of unity. Then, we can rewrite [Equation 13.2](#) as follows: **Equation 13.3**

$$Y[i] = \sum_{k=0}^{(n/2)-1} X[2k] \tilde{\omega}^{ki} + \omega^i \sum_{k=0}^{(n/2)-1} X[2k+1] \tilde{\omega}^{ki}$$

In [Equation 13.3](#), each of the two summations on the right-hand side is an $(n/2)$ -point DFT computation. If n is a power of two, each of these DFT computations can be divided similarly into smaller computations in a recursive manner. This leads to the recursive FFT algorithm given in [Algorithm 13.1](#). This FFT algorithm is called the radix-2 algorithm because at each level of recursion, the input sequence is split into two equal halves.

Algorithm 13.1 The recursive, one-dimensional, unordered, radix-2 FFT algorithm. Here $\omega = e^{2\pi\sqrt{-1}/n}$.

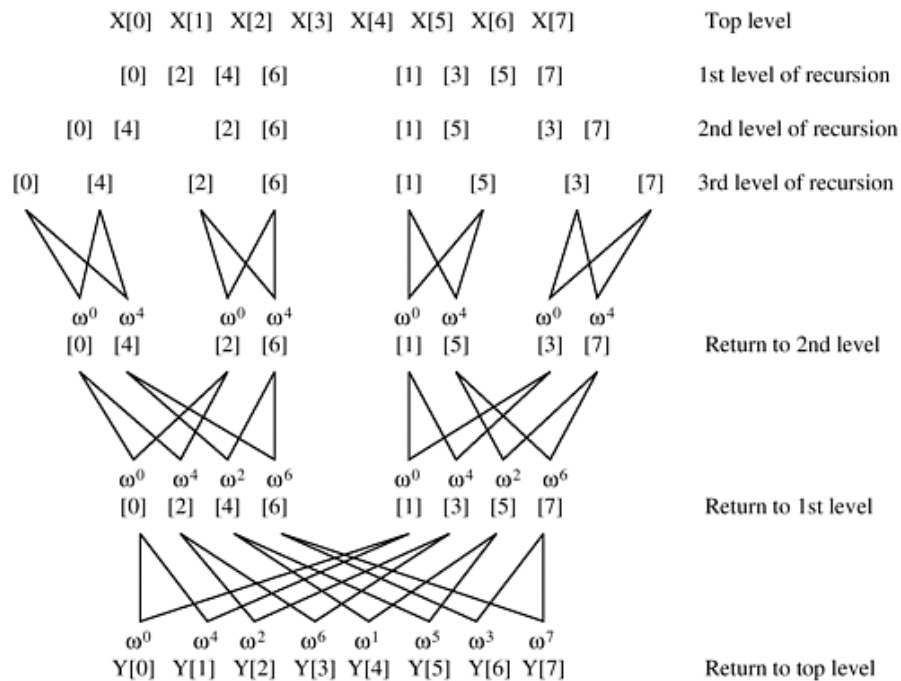
```

1.  procedure R_FFT( $X, Y, n, w$ )
2.  if ( $n = 1$ ) then  $Y[0] := X[0]$  else
3.  begin
4.    R_FFT( $\langle X[0], X[2], \dots, X[n-2] \rangle, \langle Q[0], Q[1], \dots, Q[n/2] \rangle, n/2, w^2$ );
5.    R_FFT( $\langle X[1], X[3], \dots, X[n-1] \rangle, \langle T[0], T[1], \dots, T[n/2] \rangle, n/2, w^2$ );
6.    for  $i := 0$  to  $n - 1$  do
7.       $Y[i] := Q[i \bmod (n/2)] + w^i T[i \bmod (n/2)]$ ;
8.  end R_FFT

```

Figure 13.1 illustrates how the recursive algorithm works on an 8-point sequence. As the figure shows, the first set of computations corresponding to line 7 of Algorithm 13.1 takes place at the deepest level of recursion. At this level, the elements of the sequence whose indices differ by $n/2$ are used in the computation. In each subsequent level, the difference between the indices of the elements used together in a computation decreases by a factor of two. The figure also shows the powers of w used in each computation.

Figure 13.1. A recursive 8-point unordered FFT computation.



The size of the input sequence over which an FFT is computed recursively decreases by a factor of two at each level of recursion (lines 4 and 5 of Algorithm 13.1). Hence, the maximum number of levels of recursion is $\log n$ for an initial sequence of length n . At the m th level of recursion, 2^m FFTs of size $n/2^m$ each are computed. Thus, the total number of arithmetic operations (line 7) at each level is $Q(n)$ and the overall sequential complexity of Algorithm 13.1 is $Q(n \log n)$.

The serial FFT algorithm can also be cast in an iterative form. The parallel implementations of the iterative form are easier to illustrate. Therefore, before describing parallel FFT algorithms, we give the iterative form of the serial algorithm. An iterative FFT algorithm is derived by casting each level of recursion, starting with the deepest level, as an iteration. Algorithm 13.2 gives the classic iterative Cooley-Tukey algorithm for an n -point, one-dimensional, unordered, radix-2 FFT. The program performs $\log n$ iterations of the outer loop starting on line 5. The value of the loop index m in the iterative version of the algorithm corresponds to the $(\log n - m)$ th level of recursion in the recursive version (Figure 13.1). Just as in each level of recursion, each iteration performs n complex multiplications and additions.

[Algorithm 13.2](#) has two main loops. The outer loop starting at line 5 is executed $\log n$ times for an n -point FFT, and the inner loop starting at line 8 is executed n times during each iteration of the outer loop. All operations of the inner loop are constant-time arithmetic operations. Thus, the sequential time complexity of the algorithm is $O(n \log n)$. In every iteration of the outer loop, the sequence R is updated using the elements that were stored in the sequence S during the previous iteration. For the first iteration, the input sequence X serves as the initial sequence R . The updated sequence X from the final iteration is the desired Fourier transform and is copied to the output sequence Y .

Algorithm 13.2 The Cooley-Tukey algorithm for one-dimensional, unordered, radix-2 FFT. Here $\omega = e^{2\pi\sqrt{-1}/n}$.

```

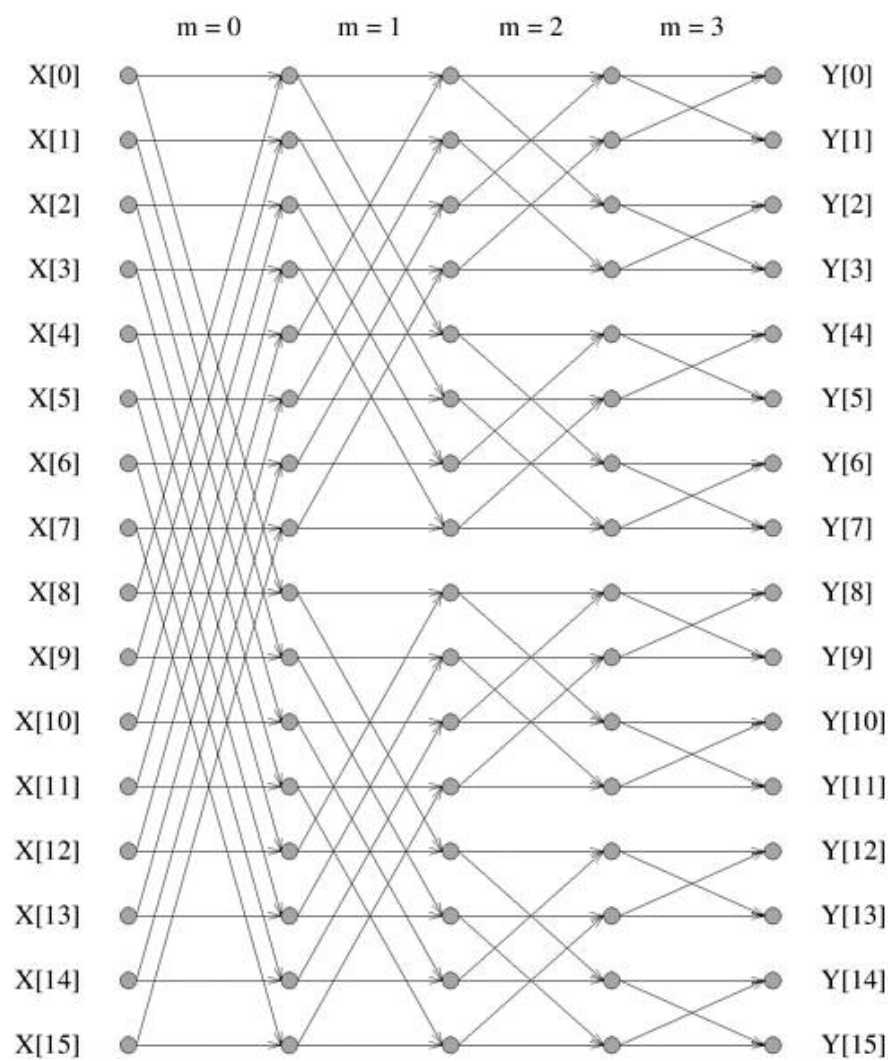
1.  procedure ITERATIVE_FFT( $X, Y, n$ )
2.  begin
3.     $r := \log n$ ;
4.    for  $i := 0$  to  $n - 1$  do  $R[i] := X[i]$ ;
5.    for  $m := 0$  to  $r - 1$  do /* Outer loop */
6.      begin
7.        for  $i := 0$  to  $n - 1$  do  $S[i] := R[i]$ ;
8.        for  $i := 0$  to  $n - 1$  do /* Inner loop */
9.          begin
10.           /* Let  $(b_0b_1 \dots b_{r-1})$  be the binary representation of  $i$  */
11.            $j := (b_0 \dots b_{m-1}0b_{m+1} \dots b_{r-1})$ ;
12.            $k := (b_0 \dots b_{m-1}1b_{m+1} \dots b_{r-1})$ ;
13.            $R[i] := S[j] + S[k] \times \omega^{(b_m b_{m-1} \dots b_0 0 \dots 0)}$ ;
14.         endfor; /* Inner loop */
15.       endfor; /* Outer loop */
16.     for  $i := 0$  to  $n - 1$  do  $Y[i] := R[i]$ ;
17.   end ITERATIVE_FFT

```

Line 12 in [Algorithm 13.2](#) performs a crucial step in the FFT algorithm. This step updates $R[i]$ by using $S[j]$ and $S[k]$. The indices j and k are derived from the index i as follows. Assume that $n = 2^r$. Since $0 \leq i < n$, the binary representation of i contains r bits. Let $(b_0b_1 \dots b_{r-1})$ be the

binary representation of index i . In the m th iteration of the outer loop ($0 \leq m < r$), index j is derived by forcing the m th most significant bit of i (that is, b_m) to zero. Index k is derived by forcing b_m to 1. Thus, the binary representations of j and k differ only in their m th most significant bits. In the binary representation of i , b_m is either 0 or 1. Hence, of the two indices j and k , one is the same as index i , depending on whether $b_m = 0$ or $b_m = 1$. In the m th iteration of the outer loop, for each i between 0 and $n - 1$, $R[i]$ is generated by executing line 12 of [Algorithm 13.2](#) on $S[i]$ and on another element of S whose index differs from i only in the m th most significant bit. [Figure 13.2](#) shows the pattern in which these elements are paired for the case in which $n = 16$.

Figure 13.2. The pattern of combination of elements of the input and the intermediate sequences during a 16-point unordered FFT computation.



13.2 The Binary-Exchange Algorithm

This section discusses the **binary-exchange algorithm** for performing FFT on a parallel computer. First, a decomposition is induced by partitioning the input or the output vector. Therefore, each task starts with one element of the input vector and computes the corresponding element of the output. If each task is assigned the same label as the index of its input or output element, then in each of the $\log n$ iterations of the algorithm, exchange of data takes place between pairs of tasks with labels differing in one bit position.

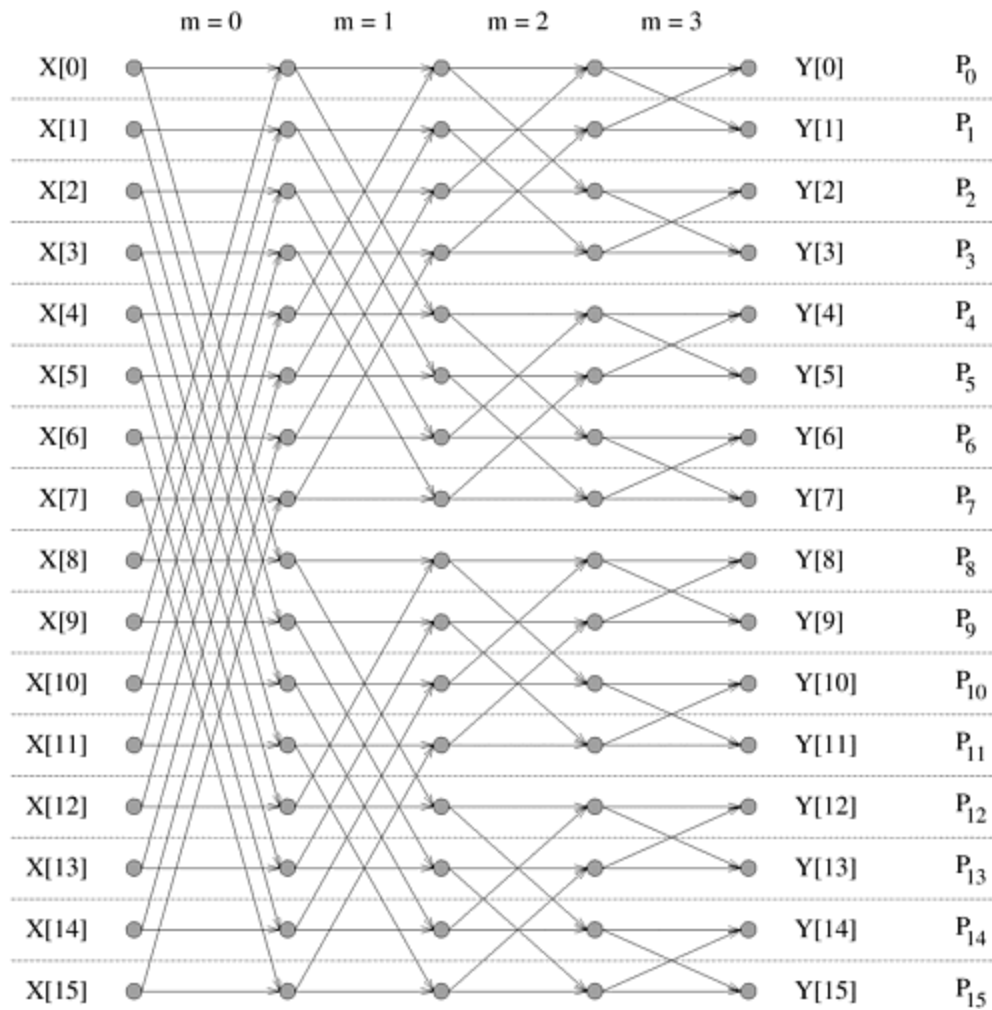
13.2.1 A Full Bandwidth Network

In this subsection, we describe the implementation of the binary-exchange algorithm on a parallel computer on which a bisection width ([Section 2.4.4](#)) of $Q(p)$ is available to p parallel processes. Since the pattern of interaction among the tasks of parallel FFT matches that of a hypercube network, we describe the algorithm assuming such an interconnection network. However, the performance and scalability analysis would be valid for any parallel computer with an overall simultaneous data-transfer capacity of $O(p)$.

One Task Per Process

We first consider a simple mapping in which one task is assigned to each process. [Figure 13.3](#) illustrates the interaction pattern induced by this mapping of the binary-exchange algorithm for $n = 16$. As the figure shows, process i ($0 \leq i < n$) initially stores $X[i]$ and finally generates $Y[i]$. In each of the $\log n$ iterations of the outer loop, process P_i updates the value of $R[i]$ by executing line 12 of [Algorithm 13.2](#). All n updates are performed in parallel.

Figure 13.3. A 16-point unordered FFT on 16 processes. P_i denotes the process labeled i .



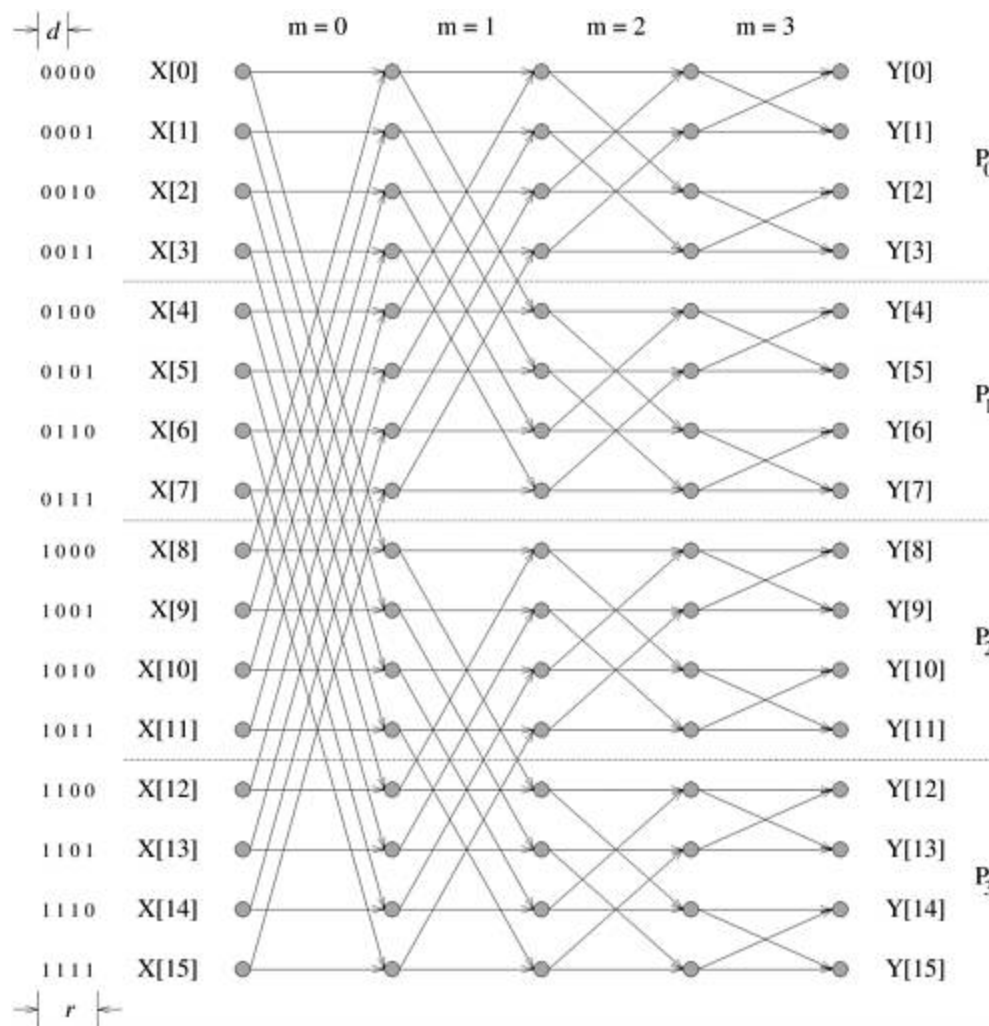
To perform the updates, process P_i requires an element of S from a process whose label differs from i at only one bit. Recall that in a hypercube, a node is connected to all those nodes whose labels differ from its own at only one bit position. Thus, the parallel FFT computation maps naturally onto a hypercube with a one-to-one mapping of processes to nodes. In the first iteration of the outer loop, the labels of each pair of communicating processes differ only at their most significant bits. For instance, processes P_0 to P_7 communicate with P_8 to P_{15} , respectively. Similarly, in the second iteration, the labels of processes communicating with each other differ at the second most significant bit, and so on.

In each of the $\log n$ iterations of this algorithm, every process performs one complex multiplication and addition, and exchanges one complex number with another process. Thus, there is a constant amount of work per iteration. Hence, it takes time $Q(\log n)$ to execute the algorithm in parallel by using a hypercube with n nodes. This hypercube formulation of FFT is cost-optimal because its process-time product is $Q(n \log n)$, the same as the complexity of a serial n -point FFT.

Multiple Tasks Per Process

We now consider a mapping in which the n tasks of an n -point FFT are mapped onto p processes, where $n > p$. For the sake of simplicity, let us assume that both n and p are powers of two, i.e., $n = 2^r$ and $p = 2^d$. As [Figure 13.4](#) shows, we partition the sequences into blocks of n/p contiguous elements and assign one block to each process.

Figure 13.4. A 16-point FFT on four processes. P_i denotes the process labeled i . In general, the number of processes is $p = 2^d$ and the length of the input sequence is $n = 2^r$.



An interesting property of the mapping shown in [Figure 13.4](#) is that, if $(b_0 b_1 \dots b_{r-1})$ is the binary representation of any i , such that $0 \leq i < n$, then $R[i]$ and $S[i]$ are mapped onto the process labeled $(b_0 \dots b_{d-1})$. That is, the d most significant bits of the index of any element of the sequence are the binary representation of the label of the process that the element belongs to. This property of the mapping plays an

important role in determining the amount of communication performed during the parallel execution of the FFT algorithm.

[Figure 13.4](#) shows that elements with indices differing at their $d (= 2)$ most significant bits are mapped onto different processes. However, all elements with indices having the same d most significant bits are mapped onto the same process. Recall from the previous section that an n -point FFT requires $r = \log n$ iterations of the outer loop. In the m th iteration of the loop, elements with indices differing in the m th most significant bit are combined. As a result, elements combined during the first d iterations belong to different processes, and pairs of elements combined during the last $(r - d)$ iterations belong to the same processes. Hence, this parallel FFT algorithm requires interprocess interaction only during the first $d = \log p$ of the $\log n$ iterations. There is no interaction during the last $r - d$ iterations. Furthermore, in the i th of the first d iterations, all the elements that a process requires come from exactly one other process $\leftrightarrow$ the one whose label is different at the i th most significant bit.

Each interaction operation exchanges n/p words of data. Therefore, the time spent in communication in the entire algorithm is $t_s \log p + t_w(n/p) \log p$. A process updates n/p elements of R during each of the $\log n$ iterations. If a complex multiplication and addition pair takes time t_c , then the parallel run time of the binary-exchange algorithm for n -point FFT on a p -node hypercube network is **Equation 13.4**

$$T_p = t_c \frac{n}{p} \log n + t_s \log p + t_w \frac{n}{p} \log p.$$

The process-time product is $t_c n \log n + t_s p \log p + t_w n \log p$. For the parallel system to be cost-optimal, this product should be $O(n \log n) \leftrightarrow$ the sequential time complexity of the FFT algorithm. This is true for $p \leq n$.

The expressions for speedup and efficiency are given by the following equations:
Equation 13.5

$$\begin{aligned} S &= \frac{t_c n \log n}{T_p} \\ &= \frac{pn \log n}{n \log n + (t_s/t_c)p \log p + (t_w/t_c)n \log p} \\ E &= \frac{1}{1 + (t_s p \log p)/(t_c n \log n) + (t_w \log p)/(t_c \log n)} \end{aligned}$$

Scalability Analysis

From [Section 13.1](#), we know that the problem size W for an n -point FFT is **Equation 13.6**

$$W = n \log n.$$

Since an n -point FFT can utilize a maximum of n processes with the mapping of [Figure 13.3](#), $n \geq p$ or $n \log n \geq p \log p$ to keep p processes busy. Thus, the isoefficiency function of this parallel FFT algorithm is $W(p \log p)$ due to concurrency. We now derive the isoefficiency function for the binary exchange algorithm due to the different communication-related terms. We can rewrite [Equation 13.5](#) as

$$\frac{t_s p \log p}{t_c n \log n} + \frac{t_w \log p}{t_c \log n} = \frac{1 - E}{E}.$$

In order to maintain a fixed efficiency E , the expression $(t_s p \log p)/(t_c n \log n) + (t_w \log p)/(t_c \log n)$ should be equal to a constant $1/K$, where $K = E/(1 - E)$. We have defined the constant K in this manner to keep the terminology consistent with [Chapter 5](#). As proposed in [Section 5.4.2](#), we use an approximation to obtain closed expressions for the isoefficiency function. We first determine the rate of growth of the problem size with respect to p that would keep the terms due to t_s constant. To do this, we assume $t_w = 0$. Now the condition for maintaining constant efficiency E is as follows: **Equation 13.7**

$$\begin{aligned} \frac{t_s p \log p}{t_c n \log n} &= \frac{1}{K} \\ n \log n &= K \frac{t_s}{t_c} p \log p \\ W &= K \frac{t_s}{t_c} p \log p \end{aligned}$$

[Equation 13.7](#) gives the isoefficiency function due to the overhead resulting from interaction latency or the message startup time.

Similarly, we derive the isoefficiency function due to the overhead resulting from t_w . We assume that $t_s = 0$; hence, a fixed efficiency E requires that the following relation be maintained: **Equation 13.8**

$$\begin{aligned} \frac{t_w \log p}{t_c \log n} &= \frac{1}{K} \\ \log n &= K \frac{t_w}{t_c} \log p \\ n &= p^{K t_w / t_c} \\ n \log n &= K \frac{t_w}{t_c} p^{K t_w / t_c} \log p \\ W &= K \frac{t_w}{t_c} p^{K t_w / t_c} \log p \end{aligned}$$

If the term Kt_w/t_c is less than one, then the rate of growth of the problem size required by [Equation 13.8](#) is less than $Q(p \log p)$. In this case, [Equation 13.7](#) determines the overall isoefficiency function of this parallel system. However, if Kt_w/t_c exceeds one, then [Equation 13.8](#) determines the overall isoefficiency function, which is now greater than the isoefficiency function of $Q(p \log p)$ given by [Equation 13.7](#).

For this algorithm, the asymptotic isoefficiency function depends on the relative values of K , t_w , and t_c . Here, K is an increasing function of the efficiency E to be maintained, t_w depends on the communication bandwidth of the parallel computer, and t_c depends on the speed of the computation speed of the processors. The FFT algorithm is unique in that the order of the isoefficiency function depends on the desired efficiency and hardware-dependent parameters. In fact, the efficiency corresponding to $Kt_w/t_c = 1$ (i.e., $1/(1 - E) = t_c/t_w$, or $E = t_c/(t_c + t_w)$) acts as a kind of threshold. For a fixed t_c and t_w , efficiencies up to the threshold can be obtained

easily. For $E \leq t_c/(t_c + t_w)$, the asymptotic isoefficiency function is $Q(p \log p)$. Efficiencies much higher than the threshold $t_c/(t_c + t_w)$ can be obtained only if the problem size is extremely large. The reason is that for these efficiencies, the asymptotic isoefficiency function is $Q(p^{Kt_w/t_c} \log p)$. The following examples illustrate the effect of the value of Kt_w/t_c on the isoefficiency function.

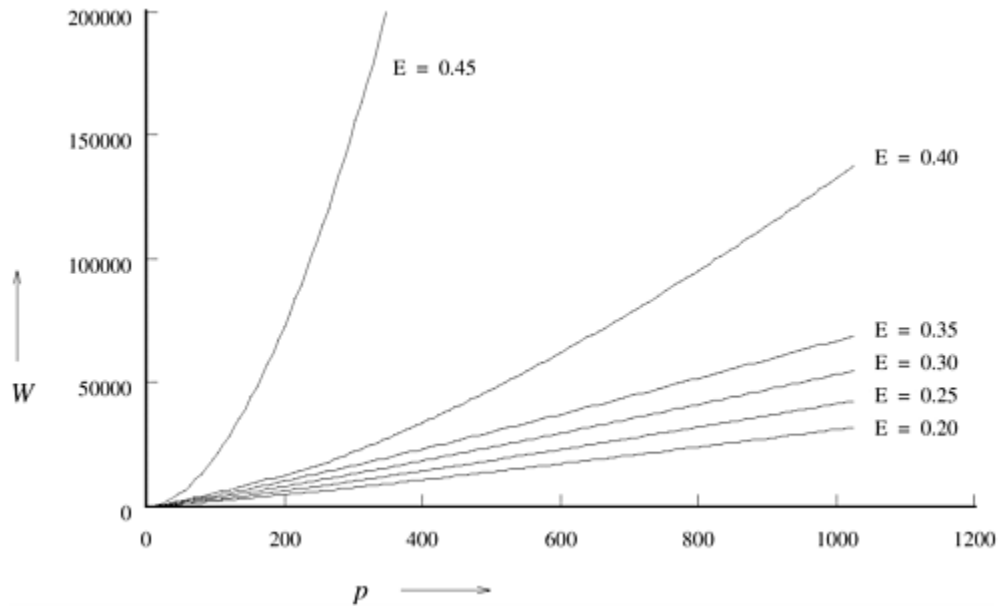
Example 13.1 Threshold effect in the binary-exchange algorithm
Consider a hypothetical hypercube for which the relative values of the hardware parameters are given by $t_c = 2$, $t_w = 4$, and $t_s = 25$. With these values, the threshold efficiency $t_c/(t_c + t_w)$ is 0.33.

Now we study the isoefficiency functions of the binary-exchange algorithm on a hypercube for maintaining efficiencies below and above the threshold. The isoefficiency function of this algorithm due to concurrency is $p \log p$. From [Equations 13.7](#) and [13.8](#), the isoefficiency functions due to the t_s and t_w terms in the overhead function are $K(t_s/t_c) p \log p$ and $K(t_w/t_c) p^{Kt_w/t_c} \log p$, respectively. To maintain a given efficiency E (that is, for a given K), the overall isoefficiency function is given

by:
$$W = \max\{p \log p, K \frac{t_s}{t_c} p \log p, K \frac{t_w}{t_c} p^{Kt_w/t_c} \log p\}$$

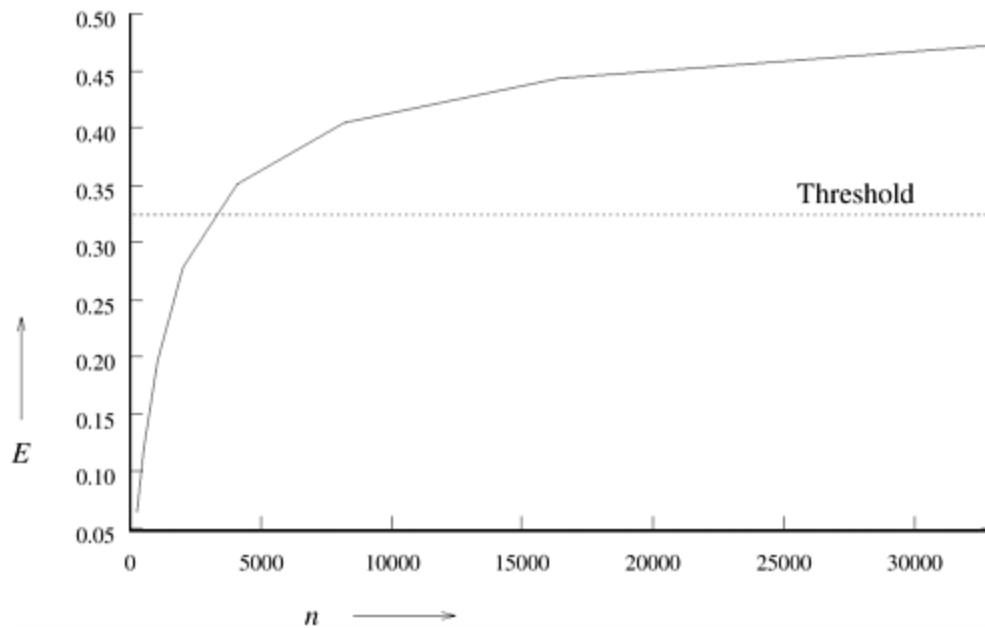
[Figure 13.5](#) shows the isoefficiency curves given by this function for $E = 0.20, 0.25, 0.30, 0.35, 0.40$, and 0.45 . Notice that the various isoefficiency curves are regularly spaced for efficiencies up to the threshold. However, the problem sizes required to maintain efficiencies above the threshold are much larger. The asymptotic isoefficiency functions for $E = 0.20, 0.25$, and 0.30 are $Q(p \log p)$. The isoefficiency function for $E = 0.40$ is $Q(p^{1.33} \log p)$, and that for $E = 0.45$ is $Q(p^{1.64} \log p)$.

Figure 13.5. Isoefficiency functions of the binary-exchange algorithm on a hypercube with $t_c = 2$, $t_w = 4$, and $t_s = 25$ for various values of E .



[Figure 13.6](#) shows the efficiency curve of n -point FFTs on a 256-node hypercube with the same hardware parameters. The efficiency E is computed by using [Equation 13.5](#) for various values of n , when p is equal to 256. The figure shows that the efficiency initially increases rapidly with the problem size, but the efficiency curve flattens out beyond the threshold. ■

Figure 13.6. The efficiency of the binary-exchange algorithm as a function of n on a 256-node hypercube with $t_c = 2$, $t_w = 4$, and $t_s = 25$.



[Example 13.1](#) shows that there is a limit on the efficiency that can be obtained for reasonable problem sizes, and that the limit is determined by the ratio between the CPU speed and the communication bandwidth of the parallel computer being used. This limit can be raised by increasing the bandwidth of the communication channels. However, making the CPUs faster without increasing the communication bandwidth lowers the limit. Hence, the binary-exchange algorithm performs poorly on a parallel computer whose communication and computation speeds are not balanced. If the hardware is balanced with respect to its communication and computation speeds, then the binary-exchange algorithm is fairly scalable, and reasonable efficiencies can be maintained while increasing the problem size at the rate of $O(p \log p)$.

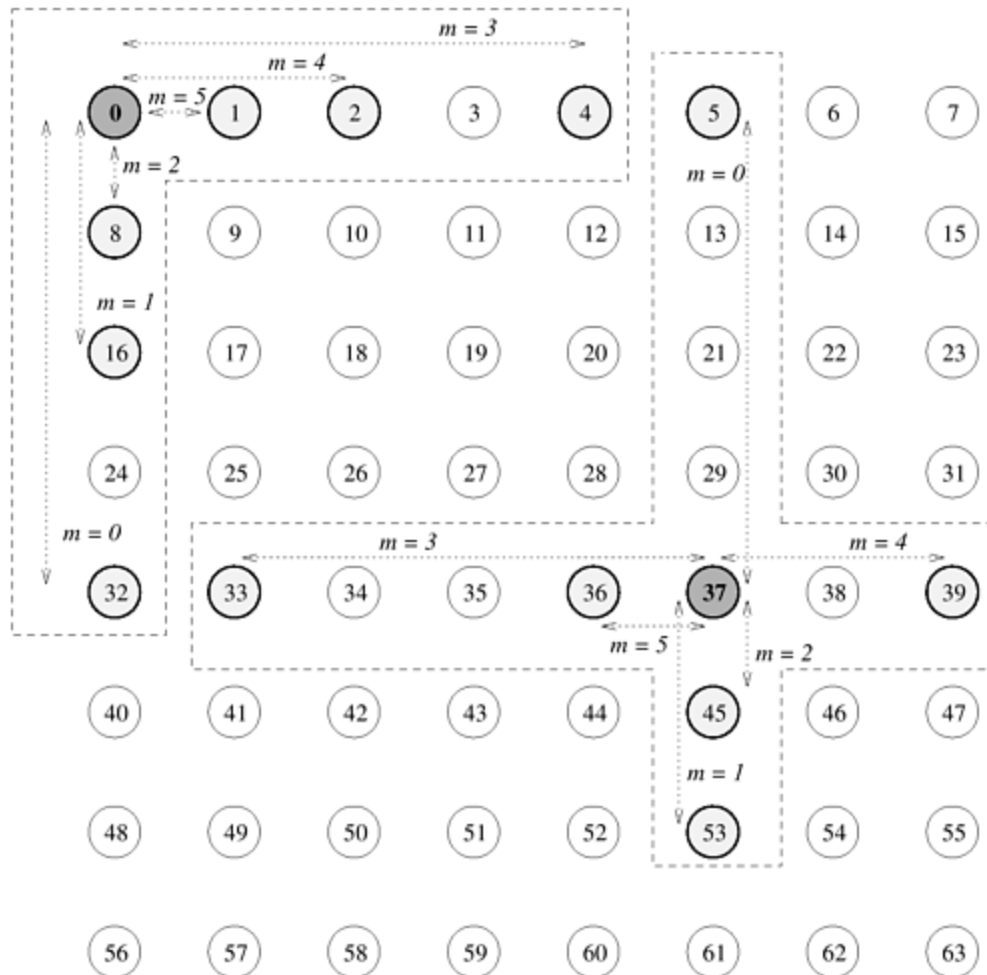
13.2.2 Limited Bandwidth Network

Now we consider the implementation of the binary-exchange algorithm on a parallel computer whose cross-section bandwidth is less than $O(p)$. We choose a mesh interconnection network to illustrate the algorithm and its performance characteristics. Assume that n tasks are mapped onto p processes running on a mesh with $\sqrt{p}$ rows and $\sqrt{p}$ columns, and that $\sqrt{p}$ is a power of two. Let $n = 2^r$ and $p = 2^d$. Also assume that the processes are labeled in a row-major fashion and that the data are distributed in the same manner as for the hypercube; that is, an element with index $(b_0 b_1 \dots b_{r-1})$ is mapped onto the process labeled $(b_0 \dots b_{d-1})$.

As in case of the hypercube, communication takes place only during the first $\log p$ iterations between processes whose labels differ at one bit. However, unlike the hypercube, the communicating processes are not directly linked in a mesh. Consequently, messages travel over multiple links and there is overlap among messages sharing the same links. [Figure 13.7](#) shows the messages sent and

received by processes 0 and 37 during an FFT computation on a 64-node mesh. As the figure shows, process 0 communicates with processes 1, 2, 4, 8, 16, and 32. Note that all these processes lie in the same row or column of the mesh as that of process 0. Processes 1, 2, and 4 lie in the same row as process 0 at distances of 1, 2, and 4 links, respectively. Processes 8, 16, and 32 lie in the same column, again at distances of 1, 2, and 4 links. More precisely, in $\log \sqrt{p}$ of the $\log p$ steps that require communication, the communicating processes are in the same row, and in the remaining $\log \sqrt{p}$ steps, they are in the same column. The number of messages that share at least one link is equal to the number of links that each message traverses (Problem 13.9) because, during a given FFT iteration, all pairs of nodes exchange messages that traverse the same number of links.

Figure 13.7. Data communication during an FFT computation on a logical square mesh of 64 processes. The figure shows all the processes with which the processes labeled 0 and 37 exchange data.



The distance between the communicating processes in a row or a column grows from one link to $\sqrt{p}/2$ links, doubling in each of the $\log \sqrt{p}$ iterations. This is true for

any process in the mesh, such as process 37 shown in [Figure 13.7](#). Thus, the total time spent in performing rowwise communication is $\sum_{m=0}^{d/2-1} (t_s + t_w(n/p)2^m)$. An equal amount of time is spent in columnwise communication. Recall that we assumed that a complex multiplication and addition pair takes time t_c . Since a process performs n/p such calculations in each of the $\log n$ iterations, the overall parallel run time is given by the following equation: **Equation 13.9**

$$\begin{aligned} T_P &= t_c \frac{n}{p} \log n + 2 \sum_{m=0}^{d/2-1} \left(t_s + t_w \frac{n}{p} 2^m \right) \\ &= t_c \frac{n}{p} \log n + 2 \left(t_s \log \sqrt{p} + t_w \frac{n}{p} (\sqrt{p} - 1) \right) \\ &\approx t_c \frac{n}{p} \log n + t_s \log p + 2t_w \frac{n}{\sqrt{p}} \end{aligned}$$

The speedup and efficiency are given by the following equations: **Equation 13.10**

$$\begin{aligned} S &= \frac{t_c n \log n}{T_P} \\ &= \frac{pn \log n}{n \log n + (t_s/t_c)p \log p + 2(t_w/t_c)n\sqrt{p}} \\ E &= \frac{1}{1 + (t_s p \log p)/(t_c n \log n) + 2(t_w \sqrt{p})/(t_c \log n)} \end{aligned}$$

The process-time product of this parallel system is $t_c n \log n + t_s p \log p + 2t_w n \sqrt{p}$. The process-time product should be $O(n \log n)$ for cost-optimality, which is obtained when $\sqrt{p} = O(\log n)$, or $p = O(\log^2 n)$. Since the communication term due to t_s in [Equation 13.9](#) is the same as for the hypercube, the corresponding isoefficiency function is again $Q(p \log p)$ as given by [Equation 13.7](#). By performing isoefficiency analysis along the same lines as in [Section 13.2.1](#), we can show that the

isoefficiency function due to the t_w term is $2K(t_w/t_c)2^{2K(t_w/t_c)\sqrt{p}}\sqrt{p}$ (Problem 13.4). Given this isoefficiency function, the problem size must grow exponentially with the number of processes to maintain constant efficiency. Hence, the binary-exchange FFT algorithm is not very scalable on a mesh.

The communication overhead of the binary-exchange algorithm on a mesh cannot be reduced by using a different mapping of the sequences onto the processes. In any mapping, there is at least one iteration in which pairs of processes that communicate with each other are at least $\sqrt{p}/2$ links apart (Problem 13.2). The algorithm inherently requires $Q(p)$ bisection bandwidth on a p -node ensemble, and on an architecture like a 2-D mesh with $Q(\sqrt{p})$ bisection bandwidth, the communication time cannot be asymptotically better than $t_s \log p + 2(n/\sqrt{p})t_w$ as discussed above.

13.2.3 Extra Computations in Parallel FFT

So far, we have described a parallel formulation of the FFT algorithm on a hypercube and a mesh, and have discussed its performance and scalability in the presence of communication overhead on both architectures. In this section, we discuss another source of overhead that can be present in a parallel FFT implementation.

Recall from [Algorithm 13.2](#) that the computation step of line 12 multiplies a power of w (a twiddle factor) with an element of S . For an n -point FFT, line 12 executes $n \log n$ times in the sequential algorithm. However, only n distinct powers of w (that is, $w^0, w^1, w^2, \dots, w^{n-1}$) are used in the entire algorithm. So some of the twiddle factors are used repeatedly. In a serial implementation, it is useful to precompute and store all n twiddle factors before starting the main algorithm. That way, the computation of twiddle factors requires only $Q(n)$ complex operations rather than the $Q(n \log n)$ operations needed to compute all twiddle factors in each iteration of line 12.

In a parallel implementation, the total work required to compute the twiddle factors cannot be reduced to $Q(n)$. The reason is that, even if a certain twiddle factor is used more than once, it might be used on different processes at different times. If FFTs of the same size are computed on the same number of processes, every process needs the same set of twiddle factors for each computation. In this case, the twiddle factors can be precomputed and stored, and the cost of their computation can be amortized over the execution of all instances of FFTs of the same size. However, if we consider only one instance of FFT, then twiddle factor computation gives rise to additional overhead in a parallel implementation, because it performs more overall operations than the sequential implementation.

As an example, consider the various powers of w used in the three iterations of an 8-point FFT. In the m th iteration of the loop starting on line 5 of the algorithm, w^l is computed for all i ($0 \leq i < n$), such that l is the integer obtained by reversing the order of the $m + 1$ most significant bits of i and then padding them by $\log n - (m + 1)$ zeros to the right (refer to [Figure 13.1](#) and [Algorithm 13.2](#) to see how l is derived). [Table 13.1](#) shows the binary representation of the powers of w required for all values of i and m for an 8-point FFT.

If eight processes are used, then each process computes and uses one column of [Table 13.1](#). Process 0 computes just one twiddle factor for all its iterations, but some processes (in this case, all other processes 2–7) compute a new twiddle factor in each of the three iterations. If $p = n/2 = 4$, then each process computes two consecutive columns of the table. In this case, the last process computes the twiddle factors in the last two columns of the table. Hence, the last process computes a total of four different powers — one each for $m = 0$ (100) and $m = 1$ (110), and two for $m = 2$ (011 and 111). Although different processes may compute a different number of twiddle factors, the total overhead due to the extra work is proportional to p times the maximum number of twiddle factors that any single process computes. Let $h(n, p)$ be the maximum number of twiddle factors that any of the p processes computes during an n -point FFT. [Table 13.2](#) shows the values of $h(8, p)$ for $p = 1, 2, 4$, and 8. The table also shows the maximum number of new twiddle factors that any single process computes in each iteration.

Table 13.1. The binary representation of the various powers of w calculated in different iterations of an 8-point FFT (also see [Figure 13.1](#)). The value of m refers to the iteration number of the outer loop, and i is the index of the inner loop of [Algorithm 13.2](#).

	<i>i</i>							
	0	1	2	3	4	5	6	7
$m = 0$	000	000	000	000	100	100	100	100
$m = 1$	000	000	100	100	010	010	110	110
$m = 2$	000	100	010	110	001	101	011	111

Table 13.2. The maximum number of new powers of w used by any process in each iteration of an 8-point FFT computation.

	$p = 1$	$p = 2$	$p = 4$	$p = 8$
$m = 0$	2	1	1	1
$m = 1$	2	2	1	1
$m = 2$	4	4	2	1
Total = $h(8, p)$	8	7	4	3

The function h is defined by the following recurrence relation (Problem 13.5):

$$h(n, 1) = n$$

$$h(p, p) = \log p \quad (p \neq 1)$$

$$h(n, p) = h(n, 2p) + n/p - 1 \quad (p \neq 1, n > p)$$

The solution to this recurrence relation for $p > 1$ and $n \geq p$ is

$$h(n, p) = 2 \left(\frac{n}{p} - 1 \right) + \log p.$$

Thus, if it takes time t'_c to compute one twiddle factor, then at least one process spends time $t'_c 2(n/p - 1) + t'_c \log p \log p$ computing twiddle factors. The total cost of twiddle factor computation, summed over all processes, is $2t'_c(n - p) + t'_c p \log p \log p$. Since even a serial implementation incurs a cost of $t'_c n$ in computing twiddle factors, the total parallel overhead due to extra work ($T_o^{extra_work}$) is given by the following

$$\begin{aligned} T_o^{extra_work} &= (2t'_c(n - p) + t'_c p \log p) - t'_c n \\ &= t'_c(n + p(\log p - 2)) \end{aligned}$$

$$\text{equation:} \quad = \Theta(n) + \Theta(p \log p)$$

This overhead is independent of the architecture of the parallel computer used for the FFT computation. The isoefficiency function due to $T_o^{extra_work}$ is $Q(p \log p)$. Since this term is of the same order as the isoefficiency terms due to message startup time and concurrency, the extra computations do not affect the overall scalability of parallel FFT.

13.3 The Transpose Algorithm

The binary-exchange algorithm yields good performance on parallel computers with sufficiently high communication bandwidth with respect to the processing speed of the CPUs. Efficiencies below a certain threshold can be maintained while increasing the problem size at a moderate rate with an increasing number of processes. However, this threshold is very low if the communication bandwidth of the parallel computer is low compared to the speed of its processors. In this section, we describe a different parallel formulation of FFT that trades off some efficiency for a more consistent level of parallel performance. This parallel algorithm involves matrix transposition, and hence is called the **transpose algorithm**.

The performance of the transpose algorithm is worse than that of the binary-exchange algorithm for efficiencies below the threshold. However, it is much easier to obtain efficiencies above the binary-exchange algorithm's threshold using the transpose algorithm. Thus, the transpose algorithm is particularly useful when the ratio of communication bandwidth to CPU speed is low and high efficiencies are desired. On a hypercube or a p -node network with $Q(p)$ bisection width, the transpose algorithm has a fixed asymptotic isoefficiency function of $Q(p^2 \log p)$. That is, the order of this isoefficiency function is independent of the ratio of the speed of point-to-point communication and the computation.

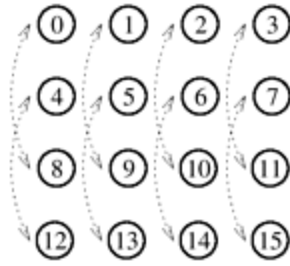
13.3.1 Two-Dimensional Transpose Algorithm

The simplest transpose algorithm requires a single transpose operation over a two-dimensional array; hence, we call this algorithm the **two-dimensional transpose algorithm**.

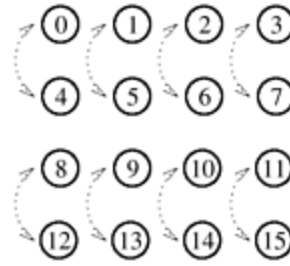
Assume that $\sqrt{n}$ is a power of 2, and that the sequences of size n used in [Algorithm 13.2](#) are arranged in a $\sqrt{n} \times \sqrt{n}$ two-dimensional square array, as shown in [Figure 13.8](#) for $n = 16$. Recall that computing the FFT of a sequence of n points requires $\log n$ iterations of the outer loop of [Algorithm 13.2](#). If the data are arranged as shown in [Figure 13.8](#), then the FFT computation in each column can proceed independently for $\log \sqrt{n}$ iterations without any column requiring data from any other column.

Similarly, in the remaining $\log \sqrt{n}$ iterations, computation proceeds independently in each row without any row requiring data from any other row. [Figure 13.8](#) shows the pattern of combination of the elements for a 16-point FFT. The figure illustrates that if data of size n are arranged in a $\sqrt{n} \times \sqrt{n}$ array, then an n -point FFT computation is equivalent to independent $\sqrt{n}$ -point FFT computations in the columns of the array, followed by independent $\sqrt{n}$ -point FFT computations in the rows.

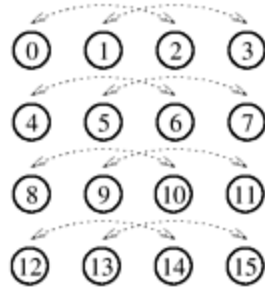
Figure 13.8. The pattern of combination of elements in a 16-point FFT when the data are arranged in a 4 x 4 two-dimensional square array.



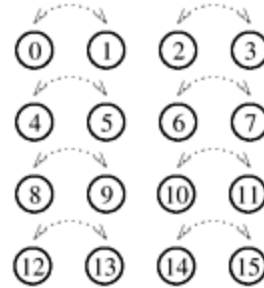
(a) Iteration $m = 0$



(b) Iteration $m = 1$



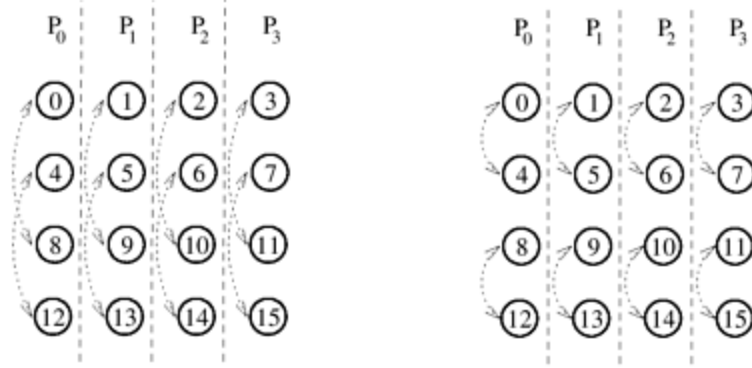
(c) Iteration $m = 2$



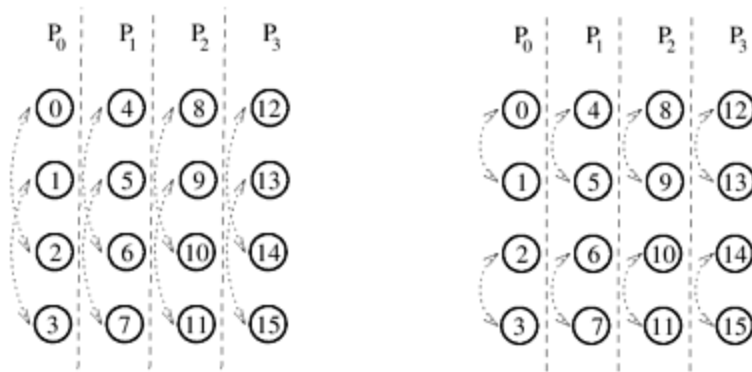
(d) Iteration $m = 3$

If the $\sqrt{n} \times \sqrt{n}$ array of data is transposed after computing the $\sqrt{n}$ -point column FFTs, then the remaining part of the problem is to compute the $\sqrt{n}$ -point columnwise FFTs of the transposed matrix. The transpose algorithm uses this property to compute the FFT in parallel by using a columnwise striped partitioning to distribute the $\sqrt{n} \times \sqrt{n}$ array of data among the processes. For instance, consider the computation of the 16-point FFT shown in [Figure 13.9](#), where the 4×4 array of data is distributed among four processes such that each process stores one column of the array. In general, the two-dimensional transpose algorithm works in three phases. In the first phase, a $\sqrt{n}$ -point FFT is computed for each column. In the second phase, the array of data is transposed. The third and final phase is identical to the first phase, and involves the computation of $\sqrt{n}$ -point FFTs for each column of the transposed array. [Figure 13.9](#) shows that the first and third phases of the algorithm do not require any interprocess communication. In both these phases, all $\sqrt{n}$ points for each columnwise FFT computation are available on the same process. Only the second phase requires communication for transposing the $\sqrt{n} \times \sqrt{n}$ matrix.

Figure 13.9. The two-dimensional transpose algorithm for a 16-point FFT on four processes.



(a) Steps in phase 1 of the transpose algorithm (before transpose)



(b) Steps in phase 3 of the transpose algorithm (after transpose)

In the transpose algorithm shown in [Figure 13.9](#), one column of the data array is assigned to one process. Before analyzing the transpose algorithm further, consider the more general case in which p processes are used and $1 \leq p \leq \sqrt{n}$. The $\sqrt{n} \times \sqrt{n}$ array of data is striped into blocks, and one block of $\sqrt{n}/p$ rows is assigned to each process. In the first and third phases of the algorithm, each process computes $\sqrt{n}/p$ FFTs of size $\sqrt{n}$ each. The second phase transposes the $\sqrt{n} \times \sqrt{n}$ matrix, which is distributed among p processes with a one-dimensional partitioning. Recall from [Section 4.5](#) that such a transpose requires an all-to-all personalized communication.

Now we derive an expression for the parallel run time of the two-dimensional transpose algorithm. The only inter-process interaction in this algorithm occurs when the $\sqrt{n} \times \sqrt{n}$ array of data partitioned along columns or rows and mapped onto p processes is transposed. Replacing the message size m by n/p in the expression for all-to-all personalized communication in [Table 4.1](#) yields $t_s(p-1) + t_w n/p$ as the time spent in the second phase of the algorithm. The first and third phases each take time $t_c \times \sqrt{n}/p \times \sqrt{n} \log \sqrt{n}$. Thus, the parallel run time of the transpose algorithm on a hypercube or any $Q(p)$ bisection width network is given by the following equation:

Equation 13.11

$$\begin{aligned}
T_P &= 2t_c \frac{\sqrt{n}}{p} \sqrt{n} \log \sqrt{n} + t_s(p-1) + t_w \frac{n}{p} \\
&= t_c \frac{n}{p} \log n + t_s(p-1) + t_w \frac{n}{p}
\end{aligned}$$

The expressions for speedup and efficiency are as follows: **Equation 13.12**

$$\begin{aligned}
S &\approx \frac{pn \log n}{n \log n + (t_s/t_c)p^2 + (t_w/t_c)n} \\
E &\approx \frac{1}{1 + (t_s p^2)/(t_c n \log n) + t_w/(t_c \log n)}
\end{aligned}$$

The process-time product of this parallel system is $t_c n \log n + t_s p^2 + t_w n$. This parallel system is cost-optimal if $n \log n = W(p^2 \log p)$.

Note that the term associated with t_w in the expression for efficiency in [Equation 13.12](#) is independent of the number of processes. The degree of concurrency of this algorithm requires that $\sqrt{n} = \Omega(p)$ because at most $\sqrt{n}$ processes can be used to partition the $\sqrt{n} \times \sqrt{n}$ array of data in a striped manner. As a result, $n = W(p^2)$, or $n \log n = W(p^2 \log p)$. Thus, the problem size must increase at least as fast as $Q(p^2 \log p)$ with respect to the number of processes to use all of them efficiently. The overall isoefficiency function of the two-dimensional transpose algorithm is $Q(p^2 \log p)$ on a hypercube or another interconnection network with bisection width $Q(p)$. This isoefficiency function is independent of the ratio of t_w for point-to-point communication and t_c . On a network whose cross-section bandwidth b is less than $Q(p)$ for p nodes, the t_w term must be multiplied by an appropriate expression of $Q(p/b)$ in order to derive T_P , S , E , and the isoefficiency function (Problem 13.6).

Comparison with the Binary-Exchange Algorithm

A comparison of [Equations 13.4](#) and [13.11](#) shows that the transpose algorithm has a much higher overhead than the binary-exchange algorithm due to the message startup time t_s , but has a lower overhead due to per-word transfer time t_w . As a result, either of the two algorithms may be faster depending on the relative values of t_s and t_w . If the latency t_s is very low, then the transpose algorithm may be the algorithm of choice. On the other hand, the binary-exchange algorithm may perform better on a parallel computer with a high communication bandwidth but a significant startup time.

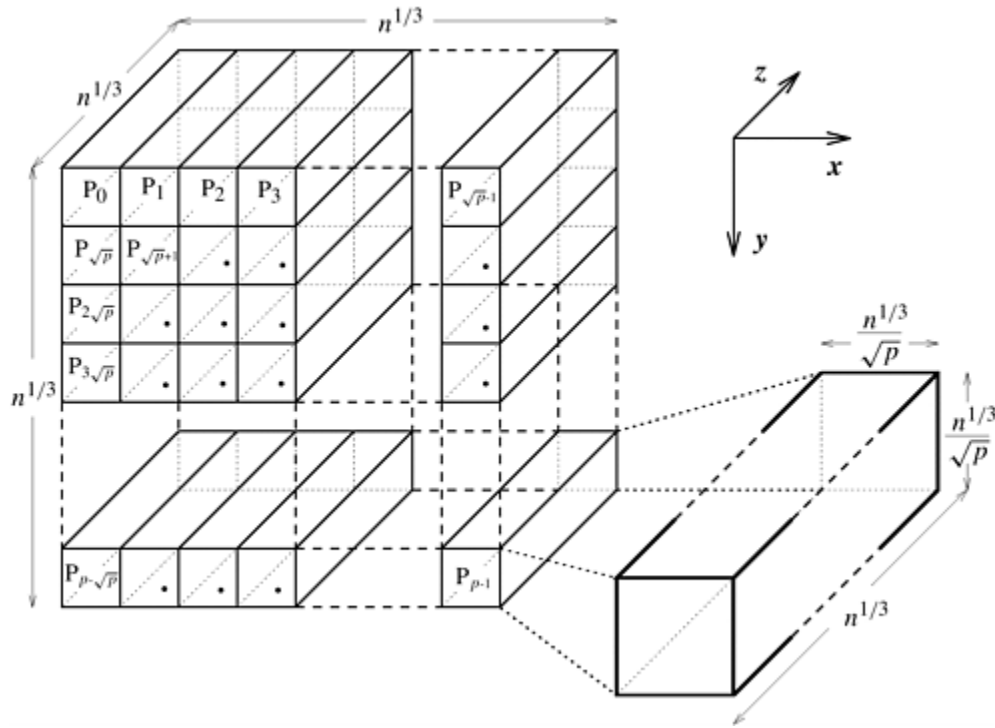
Recall from [Section 13.2.1](#) that an overall isoefficiency function of $Q(p \log p)$ can be realized by using the binary-exchange algorithm if the efficiency is such that $K t_w / t_c \leq 1$, where $K = E / (1 - E)$. If the desired efficiency is such that $K t_w / t_c = 2$, then the

overall isoefficiency function of both the binary-exchange and the two-dimensional transpose algorithms is $Q(p^2 \log p)$. When $Kt_w/t_c > 2$, the two-dimensional transpose algorithm is more scalable than the binary-exchange algorithm; hence, the former should be the algorithm of choice, provided that $n \geq p^2$. Note, however, that the transpose algorithm yields a performance benefit over the binary-exchange algorithm only if the target architecture has a cross-section bandwidth of $Q(p)$ for p nodes (Problem 13.6).

13.3.2 The Generalized Transpose Algorithm

In the two-dimensional transpose algorithm, the input of size n is arranged in a $\sqrt{n} \times \sqrt{n}$ two-dimensional array that is partitioned along one dimension on p processes. These processes, irrespective of the underlying architecture of the parallel computer, can be regarded as arranged in a logical one-dimensional linear array. As an extension of this scheme, consider the n data points to be arranged in an $n^{1/3} \times n^{1/3} \times n^{1/3}$ three-dimensional array mapped onto a logical $\sqrt{p} \times \sqrt{p}$ two-dimensional mesh of processes. [Figure 13.10](#) illustrates this mapping. To simplify the algorithm description, we label the three axes of the three-dimensional array of data as x , y , and z . In this mapping, the x - y plane of the array is checkerboarded into $\sqrt{p} \times \sqrt{p}$ parts. As the figure shows, each process stores $(n^{1/3}/\sqrt{p}) \times (n^{1/3}/\sqrt{p})$ columns of data, and the length of each column (along the z -axis) is $n^{1/3}$. Thus, each process has $(n^{1/3}/\sqrt{p}) \times (n^{1/3}/\sqrt{p}) \times n^{1/3} = n/p$ elements of data.

Figure 13.10. Data distribution in the three-dimensional transpose algorithm for an n -point FFT on p processes ($\sqrt{p} \leq n^{1/3}$).



Recall from [Section 13.3.1](#) that the FFT of a two-dimensionally arranged input of size $\sqrt{n} \times \sqrt{n}$ can be computed by first computing the $\sqrt{n}$ -point one-dimensional FFTs of all the columns of the data and then computing the $\sqrt{n}$ -point one-dimensional FFTs of all the rows. If the data are arranged in an $n^{1/3} \times n^{1/3} \times n^{1/3}$ three-dimensional array, the entire n -point FFT can be computed similarly. In this case, $n^{1/3}$ -point FFTs are computed over the elements of the columns of the array in all three dimensions, choosing one dimension at a time. We call this algorithm the **three-dimensional transpose algorithm**. This algorithm can be divided into the following five phases:

1. In the first phase, $n^{1/3}$ -point FFTs are computed on all the rows along the z -axis.
2. In the second phase, all the $n^{1/3}$ cross-sections of size $n^{1/3} \times n^{1/3}$ along the y - z plane are transposed.
3. In the third phase, $n^{1/3}$ -point FFTs are computed on all the rows of the modified array along the z -axis.
4. In the fourth phase, each of the $n^{1/3} \times n^{1/3}$ cross-sections along the x - z plane is transposed.
5. In the fifth and final phase, $n^{1/3}$ -point FFTs of all the rows along the z -axis are computed again.

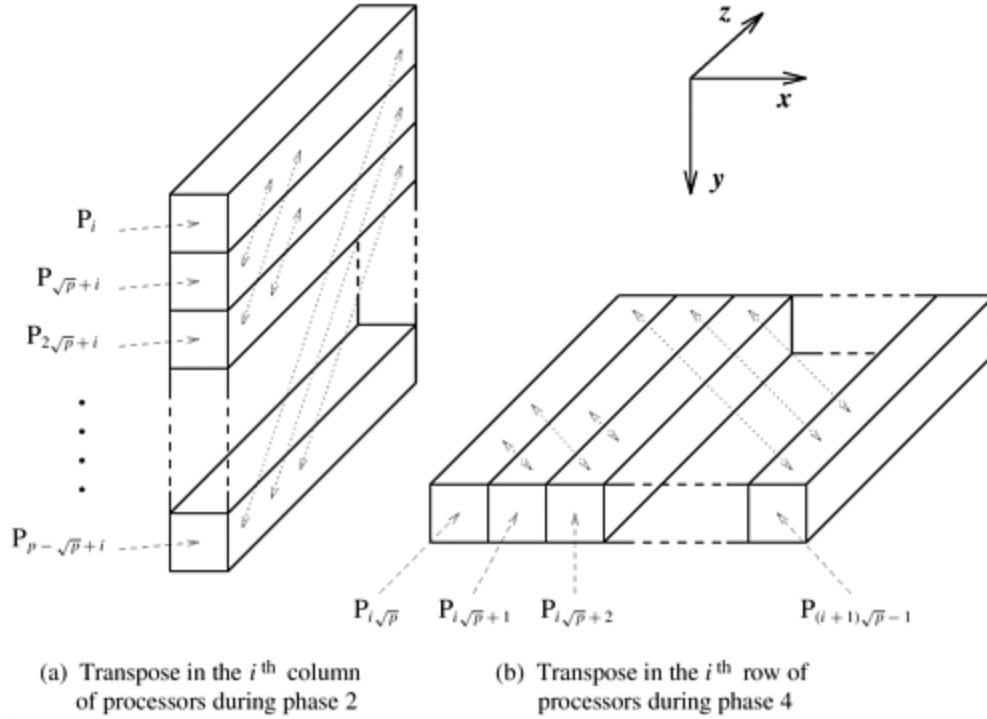
For the data distribution shown in [Figure 13.10](#), in the first, third, and fifth phases of the algorithm, all processes perform $(n^{1/3}/\sqrt{p}) \times (n^{1/3}/\sqrt{p})$ FFT computations, each of size $n^{1/3}$. Since all the data for performing these computations are locally available

on each process, no interprocess communication is involved in these three odd-numbered phases. The time spent by a process in each of these phases is $t_c n^{1/3} \log(n^{1/3}) \times (n^{1/3}/\sqrt{p}) \times (n^{1/3}/\sqrt{p})$. Thus, the total time that a process spends in computation is $t_c(n/p) \log n$.

[Figure 13.11](#) illustrates the second and fourth phases of the three-dimensional transpose algorithm. As [Figure 13.11\(a\)](#) shows, the second phase of the algorithm requires transposing square cross-sections of size $n^{1/3} \times n^{1/3}$ along the y - z plane. Each column of $\sqrt{p}$ processes performs the transposition of $(n^{1/3}/\sqrt{p})$ such cross-sections. This transposition involves all-to-all personalized communications among groups of $\sqrt{p}$ processes with individual messages of size $n/p^{3/2}$. If a p -node network with bisection width $Q(p)$ is used, this phase takes time $t_s(\sqrt{p} - 1) + t_w n/p$. The fourth phase, shown in [Figure 13.11\(b\)](#), is similar. Here each row of $\sqrt{p}$ processes performs the transpose of $(n^{1/3}/\sqrt{p})$ cross-sections along the x - z plane. Again, each cross-section consists of $n^{1/3} \times n^{1/3}$ data elements. The communication time of this phase is the same as that of the second phase. The total parallel run time of the three-dimensional transpose algorithm for an n -point FFT is **Equation 13.13**

$$T_P = t_c \frac{n}{p} \log n + 2t_s(\sqrt{p} - 1) + 2t_w \frac{n}{p}.$$

Figure 13.11. The communication (transposition) phases in the three-dimensional transpose algorithm for an n -point FFT on p processes.



Having studied the two- and three-dimensional transpose algorithms, we can derive a more general q -dimensional transpose algorithm similarly. Let the n -point input be arranged in a logical q -dimensional array of size $n^{1/q} \times n^{1/q} \times \dots \times n^{1/q}$ (a total of q terms). Now the entire n -point FFT computation can be viewed as q subcomputations. Each of the q subcomputations along a different dimension consists of $n^{(q-1)/q}$ FFTs over $n^{1/q}$ data points. We map the array of data onto a logical $(q-1)$ -dimensional array of p processes, where $p \leq n^{(q-1)/q}$, and $p = 2^{(q-1)s}$ for some integer s . The FFT of the entire data is now computed in $(2q-1)$ phases (recall that there are three phases in the two-dimensional transpose algorithm and five phases in the three-dimensional transpose algorithm). In the q odd-numbered phases, each process performs $n^{(q-1)/q}/p$ of the required $n^{1/q}$ -point FFTs. The total computation time for each process over all q computation phases is the product of q (the number of computation phases), $n^{(q-1)/q}/p$ (the number of $n^{1/q}$ -point FFTs computed by each process in each computation phase), and $t_c n^{1/q} \log(n^{1/q})$ (the time to compute a single $n^{1/q}$ -point FFT). Multiplying these terms gives a total computation time of $t_c(n/p) \log n$.

In each of the $(q-1)$ even-numbered phases, sub-arrays of size $n^{1/q} \times n^{1/q}$ are transposed on rows of the q -dimensional logical array of processes. Each such row contains $p^{1/(q-1)}$ processes. One such transpose is performed along every dimension of the $(q-1)$ -dimensional process array in each of the $(q-1)$ communication phases. The time spent in communication in each transposition is $t_s(p^{1/(q-1)} - 1) + t_w n/p$. Thus, the total parallel run time of the q -dimensional transpose algorithm for an n -point FFT on a p -node network with bisection width $Q(p)$ is **Equation 13.14**

$$T_P = t_c \frac{n}{p} \log n + (q-1)t_s(p^{1/(q-1)} - 1) + (q-1)t_w \frac{n}{p}.$$

[Equation 13.14](#) can be verified by replacing q with 2 and 3, and comparing the result with Equations [13.11](#) and [13.13](#), respectively.

A comparison of Equations [13.11](#), [13.13](#), [13.14](#), and [13.4](#) shows an interesting trend. As the dimension q of the transpose algorithm increases, the communication overhead due to t_w increases, but that due to t_s decreases. The binary-exchange algorithm and the two-dimensional transpose algorithms can be regarded as two extremes. The former minimizes the overhead due to t_s but has the largest overhead due to t_w . The latter minimizes the overhead due to t_w but has the largest overhead due to t_s . The variations of the transpose algorithm for $2 < q < \log p$ lie between these two extremes. For a given parallel computer, the specific values of t_c , t_s , and t_w determine which of these algorithms has the optimal parallel run time (Problem 13.8).

Note that, from a practical point of view, only the binary-exchange algorithm and the two- and three-dimensional transpose algorithms are feasible. Higher-dimensional transpose algorithms are very complicated to code. Moreover, restrictions on n and p limit their applicability. These restrictions for a q -dimensional transpose algorithm are that n must be a power of two that is a multiple of q , and that p must be a power of 2 that is a multiple of $(q-1)$. In other words, $n = 2^{qr}$, and $p = 2^{(q-1)s}$, where q , r , and s are integers.

Example 13.2 A comparison of binary-exchange, 2-D transpose, and 3-D transpose algorithms This example shows that either the binary-exchange algorithm or any of the transpose algorithms may be the algorithm of choice for a given parallel computer, depending on the size of the FFT. Consider a 64-node version of the hypercube described in [Example 13.1](#) with $t_c = 2$, $t_s = 25$, and $t_w = 4$. [Figure 13.12](#) shows speedups attained by the binary-exchange algorithm, the 2-D transpose algorithm, and the 3-D transpose algorithm for different problem sizes. The speedups are based on the parallel run times given by Equations [13.4](#), [13.11](#), and [13.13](#), respectively. The figure shows that for different ranges of n , a different algorithm provides the highest speedup for an n -point FFT. For the given values of the hardware parameters, the binary-exchange algorithm is best suited for very low granularity FFT computations, the 2-D transpose algorithm is best for very high granularity computations, and the 3-D transpose algorithm's speedup is the maximum for intermediate granularities. ■

Figure 13.12. A comparison of the speedups obtained by the binary-exchange, 2-D transpose, and 3-D transpose algorithms on

a 64-node hypercube with $t_c = 2$, $t_w = 4$, and $t_s = 25$.

